

TOTAL SECTIONS	CURRICULUM PART	CORE MECHANICS	REFERENCE STANDARD
92 Modules + Practice	Part 07 — Core Built-in Standard Library	Date, Math, Number, BigInt, Intl & RegExp	ECMAScript 2026 / V8 Runtime

Built-in Objects: Date, Math, Number, BigInt, Intl & RegExp

Chapter 12-এ আমরা **Map**, **Set**, **WeakMap**, **WeakSet** শিখেছি। এবার JavaScript-এর built-in tools/objects শিখব। এগুলো practically প্রতিদিনের JavaScript development-এ কাজে লাগে:

Date • Math • Number • BigInt • Intl • RegExp • String Utilities

তারিখ, গাণিতিক হিসাব, বড় সংখ্যা ও আন্তর্জাতিক ফরম্যাটিংয়ের ভিত্তি মজবুত করতে এই অধ্যায়টি অত্যন্ত গুরুত্বপূর্ণ। 🔥

PART 01 — DATE OBJECT: TIME, CALCULATIONS & INTL.DATETIMEFORMAT (13.1 – 13.20)

13.1 Built-in Object কী?

JavaScript আমাদের জন্য আগে থেকেই কিছু object/API তৈরি করে দিয়েছে।

যেমন:

13.1 JAVASCRIPT

Math
Date
Number
String
RegExp
JSON
Object
Array
Map
Set

RUN

এগুলোকে আমরা নিজেরা শুরু থেকে তৈরি না করেও ব্যবহার করতে পারি।

💡 REAL-LIFE EXAMPLE

একটি e-commerce website-এ দরকার হতে পারে:

Product price calculation → Math / Number
Order date → Date
Currency formatting → Intl
Search validation → RegExp
Large transaction ID → BigInt

13.2 Date Object

JavaScript-এ date এবং time নিয়ে কাজ করার জন্য:

13.2 JAVASCRIPT

RUN

Date

ব্যবহার করা হয়।

Current Date & Time

13.2 JAVASCRIPT

RUN

```
const now = new Date();  
  
console.log(now);
```

Output তোমার computer-এর current date/time অনুযায়ী হবে।

উদাহরণ:

TEXT

Wed Sep 16 2026 14:30:00 GMT+0600

13.3 Date.now()

বর্তমান সময়ের timestamp পাওয়া যায়:

13.3 JAVASCRIPT

RUN

```
console.log(Date.now());
```

Output এরকম হতে পারে:

TEXT

1789545600000

এটি হলো:

Unix Timestamp / milliseconds since January 1, 1970 UTC

13.4 Date Object-এর বিভিন্ন অংশ

13.4 JAVASCRIPT

RUN

```
const date = new Date();  
  
console.log(date.getFullYear());  
console.log(date.getMonth());  
console.log(date.getDate());  
console.log(date.getDay());  
console.log(date.getHours());  
console.log(date.getMinutes());  
console.log(date.getSeconds());
```

13.5 getFullYear()

13.5 JAVASCRIPT

RUN

```
const date = new Date();  
  
console.log(date.getFullYear());
```

TEXT

2026

13.6 `getMonth()` — গুরুত্বপূর্ণ ⚠️

13.6 JAVASCRIPT

RUN

```
const date = new Date();  
  
console.log(date.getMonth());
```

`getMonth()` 0-based।

```
January   → 0  
February  → 1  
March     → 2  
...  
December  → 11
```

তাই:

13.6 JAVASCRIPT

RUN

```
const month = new Date().getMonth() + 1;
```

ব্যবহার করলে সাধারণ 1-12 month number পাওয়া যায়।

13.7 `getDate()`

মাসের কত তারিখ:

13.7 JAVASCRIPT

RUN

```
const date = new Date();  
  
console.log(date.getDate());
```

যদি 16 September হয়:

TEXT

16

13.8 `getDay()`

সপ্তাহের কোন দিন:

13.8 JAVASCRIPT

RUN

```
const date = new Date();  
  
console.log(date.getDay());
```

এটিও 0-based:

```
Sunday    → 0  
Monday    → 1  
Tuesday   → 2  
Wednesday → 3  
Thursday  → 4  
Friday    → 5  
Saturday  → 6
```

13.9 Time Methods

13.9 JAVASCRIPT

RUN

```
const date = new Date();  
  
console.log(date.getHours());  
console.log(date.getMinutes());  
console.log(date.getSeconds());  
console.log(date.getMilliseconds());
```

13.10 Date থেকে সুন্দরভাবে Date বের করা

13.10 JAVASCRIPT

RUN

```
const date = new Date();

const year = date.getFullYear();
const month = date.getMonth() + 1;
const day = date.getDate();

console.log(`${day}/${month}/${year}`);
```

Example:

TEXT

16/9/2026

13.11 Specific Date তৈরি করা

13.11 JAVASCRIPT

RUN

```
const date = new Date("2026-09-16");

console.log(date);
```

13.12 Date Constructor

আরেকভাবে:

13.12 JAVASCRIPT

RUN

```
const date = new Date(2026, 8, 16);

console.log(date);
```

⚠ এখানে month:

TEXT

0 = January
1 = February
...
8 = September

তাই:

13.12 JAVASCRIPT

RUN

```
new Date(2026, 8, 16)
```

মানে:

TEXT

16 September 2026

13.13 Date Comparison

Date object compare করা যায়।

13.13 JAVASCRIPT

RUN

```
const date1 = new Date("2026-09-10");
const date2 = new Date("2026-09-20");

console.log(date1 < date2);
```

TEXT

true

13.14 Date Difference

ধরো:

13.14 JAVASCRIPT

RUN

```
const start = new Date("2026-09-01");
const end = new Date("2026-09-16");

const difference = end - start;

console.log(difference);
```

এখানে result milliseconds-এ আসবে।

Days বের করতে:

13.14 JAVASCRIPT

RUN

```
const days = difference / (1000 * 60 * 60 * 24);

console.log(days);
```

TEXT

15

13.15 Real-Life Example — Internship Duration

13.15 JAVASCRIPT

RUN

```
const start = new Date("2026-09-10");
const end = new Date("2026-10-10");

const difference = end - start;

const days = difference / (1000 * 60 * 60 * 24);

console.log(`Duration: ${days} days`);
```

TEXT

Duration: 30 days

13.16 Date Setter Methods

Date-এর value পরিবর্তন করা যায়।

13.16 JAVASCRIPT

RUN

```
const date = new Date();

date.setFullYear(2030);
date.setMonth(5);
date.setDate(20);

console.log(date);
```

Common methods:

```
setFullYear()
setMonth()
setDate()
setHours()
setMinutes()
setSeconds()
```

13.17 UTC Methods

JavaScript-এ local time এবং UTC time আলাদা হতে পারে।

যেমন:

13.17 JAVASCRIPT

RUN

```
const date = new Date();  
  
console.log(date.getHours());  
console.log(date.getUTCHours());
```

`getHours()` → local time

`getUTCHours()` → UTC time

13.18 `toISOString()`

API এবং database-এর জন্য খুব usefull

13.18 JAVASCRIPT

RUN

```
const date = new Date();  
  
console.log(date.toISOString());
```

TEXT

2026-09-16T08:30:00.000Z

ISO format সাধারণত API communication-এ অনেক ব্যবহৃত হয়।

13.19 `toLocaleDateString()`

User-friendly date:

13.19 JAVASCRIPT

RUN

```
const date = new Date();  
  
console.log(  
  date.toLocaleDateString()  
);
```

Output locale অনুযায়ী পরিবর্তিত হতে পারে।

13.20 `Intl.DateTimeFormat`

Professional date formatting-এর জন্য:

13.20 JAVASCRIPT

RUN

```
const date = new Date("2026-09-16");  
  
const formatter = new Intl.DateTimeFormat(  
  "en-US"  
);  
  
console.log(formatter.format(date));
```

TEXT

9/16/2026

13.21 Math Object

Math object mathematical operations-এর জন্য।

13.21 JAVASCRIPT

RUN

```
Math
```

এর অনেক useful method আছে।

13.22 Math.PI

13.22 JAVASCRIPT

RUN

```
console.log(Math.PI);
```

TEXT

3.141592653589793

13.23 Math.E

13.23 JAVASCRIPT

RUN

```
console.log(Math.E);
```

TEXT

2.718281828459045

13.24 Math.round()

নিকটবর্তী integer:

13.24 JAVASCRIPT

RUN

```
console.log(Math.round(4.4));  
console.log(Math.round(4.6));
```

TEXT

4
5**13.25** Math.floor()

নিচের দিকে round:

13.25 JAVASCRIPT

RUN

```
console.log(Math.floor(4.9));
```

TEXT

4

13.26 Math.ceil()

উপরের দিকে round:

13.26 JAVASCRIPT

RUN

```
console.log(Math.ceil(4.1));
```

TEXT

5

13.27 `Math.trunc()`

Decimal অংশ বাদ দেয়:

13.27 JAVASCRIPT

RUN

```
console.log(Math.trunc(4.99));
```

TEXT

4

Negative number-ও:

13.27 JAVASCRIPT

RUN

```
console.log(Math.trunc(-4.99));
```

TEXT

-4

13.28 `round()` বনাম `floor()` বনাম `ceil()` বনাম `trunc()`

Method	4.7	-4.7
<code>round()</code>	5	-5
<code>floor()</code>	4	-5
<code>ceil()</code>	5	-4
<code>trunc()</code>	4	-4

13.29 `Math.abs()`

Absolute value:

13.29 JAVASCRIPT

RUN

```
console.log(Math.abs(-50));
```

TEXT

50

13.30 `Math.pow()`

Power:

13.30 JAVASCRIPT

RUN

```
console.log(Math.pow(2, 3));
```

TEXT

8

তবে modern JavaScript-ও:

13.30 JAVASCRIPT

RUN

```
console.log(2 ** 3);
```

এটিও ব্যবহার করা যায়।

13.31 Math.sqrt()

Square root:

13.31 JAVASCRIPT

RUN

```
console.log(Math.sqrt(64));
```

TEXT

8

13.32 Math.cbrt()

Cube root:

13.32 JAVASCRIPT

RUN

```
console.log(Math.cbrt(27));
```

TEXT

3

13.33 Math.max()

সবচেয়ে বড় number:

13.33 JAVASCRIPT

RUN

```
console.log(Math.max(10, 50, 20, 80));
```

TEXT

80

13.34 Math.min()

সবচেয়ে ছোট:

13.34 JAVASCRIPT

RUN

```
console.log(Math.min(10, 50, 20, 80));
```

TEXT

10

13.35 Array থেকে Maximum

13.35 JAVASCRIPT

RUN

```
const numbers = [10, 50, 20, 80];  
  
console.log(Math.max(... numbers));
```

TEXT

80

এখানে:

13.35 JAVASCRIPT

RUN

```
... numbers
```

spread operator।

13.36

Math.random()

Random number:

13.36 JAVASCRIPT

RUN

```
console.log(Math.random());
```

TEXT

0.7348234

প্রতিবার আলাদা value আসবে।

Range:

```
0 <= Math.random() < 1
```

13.37

Random Integer 1–10

13.37 JAVASCRIPT

RUN

```
const random = Math.floor(  
  Math.random() * 10  
) + 1;  
  
console.log(random);
```

Possible:

TEXT

```
1  
2  
3  
...  
10
```

13.38

Random Integer Function

13.38 JAVASCRIPT

RUN

```
function randomNumber(min, max) {  
  return Math.floor(  
    Math.random() * (max - min + 1)  
  ) + min;  
}  
  
console.log(randomNumber(1, 100));
```

Output হতে পারে:

TEXT

47

13.39 Real-Life Example — OTP-like Number

13.39 JAVASCRIPT

RUN

```
function generateCode() {  
  return Math.floor(  
    100000 + Math.random() * 900000  
  );  
}  
  
console.log(generateCode());
```

Possible:

TEXT

583214

⚠ Security-sensitive authentication/OTP-এর জন্য সাধারণ `Math.random()` ব্যবহার করা উচিত নয়। সেফ্রেব্র browser-এর cryptographic random APIs ব্যবহার করতে হয়।

PART 03 — NUMBER & BIGINT: SAFE INTEGERS, PRECISION & LARGE NUMBERS (13.40 – 13.56)

13.40 `Number` Object

JavaScript-এ numbers নিয়ে কাজ করার জন্য `Number` ব্যবহার করা হয়।

13.40 JAVASCRIPT

RUN

```
const price = 999;  
  
console.log(typeof price);
```

number

13.41 `Number.isInteger()`

Integer কিনা:

13.41 JAVASCRIPT

RUN

```
console.log(Number.isInteger(10));  
console.log(Number.isInteger(10.5));
```

TEXT

true
false

13.42 `Number.isNaN()`

13.42 JAVASCRIPT

RUN

```
console.log(Number.isNaN(NaN));  
console.log(Number.isNaN("hello"));
```

TEXT

true
false

এটি strict এবং predictable।

13.43

NaN

NaN | মানে:

Not-a-Number

Example:

13.43 JAVASCRIPT

RUN

```
const result = "hello" / 2;  
  
console.log(result);
```

TEXT

NaN

13.44

Number.isFinite()

13.44 JAVASCRIPT

RUN

```
console.log(Number.isFinite(100));  
console.log(Number.isFinite(Infinity));  
console.log(Number.isFinite("100"));
```

TEXT

```
true  
false  
false
```

13.45

Number.MAX_VALUE

JavaScript Number-এর খুব বড় finite value:

13.45 JAVASCRIPT

RUN

```
console.log(Number.MAX_VALUE);
```

13.46

Number.MIN_VALUE

সবচেয়ে ছোট positive non-zero number-এর কাছাকাছি:

13.46 JAVASCRIPT

RUN

```
console.log(Number.MIN_VALUE);
```

⚠ এটি "সবচেয়ে ছোট negative number" নয়। এটি smallest positive non-zero floating-point value-এর concept।

13.47

Number.MAX_SAFE_INTEGER

এটি খুব গুরুত্বপূর্ণ।

13.47 JAVASCRIPT

RUN

```
console.log(Number.MAX_SAFE_INTEGER);
```

TEXT

9007199254740991

এর পরে integer precision issue হতে পারে।

13.48 Safe Integer

13.48 JAVASCRIPT

RUN

```
console.log(  
  Number.isSafeInteger(9007199254740991)  
);
```

TEXT

true

13.49 Unsafe Integer Problem

13.49 JAVASCRIPT

RUN

```
console.log(  
  9007199254740991 + 1  
);  
  
console.log(  
  9007199254740991 + 2  
);
```

Large integer precision সীমার কারণে surprising behavior হতে পারে।

এখানে আসে:

BigInt

13.50 BigInt কী?

BigInt ব্যবহার করা হয় খুব বড় integer-এর জন্য।

Syntax:

13.50 JAVASCRIPT

RUN

```
const bigNumber = 123456789012345678901234567890n;
```

শেষে:

TEXT

n

দিতে হয়।

13.51 BigInt Example

13.51 JAVASCRIPT

RUN

```
const bigNumber =  
  123456789012345678901234567890n;  
  
console.log(bigNumber);
```

TEXT

123456789012345678901234567890n

13.52 BigInt Type

13.52 JAVASCRIPT

RUN

```
const bigNumber = 123n;  
  
console.log(typeof bigNumber);
```

bigint

13.53 BigInt Calculation

13.53 JAVASCRIPT

RUN

```
const a = 1000000000000000000n;  
const b = 2000000000000000000n;  
  
console.log(a + b);
```

TEXT

3000000000000000000n

13.54 Number + BigInt

এভাবে করা যাবে না:

13.54 JAVASCRIPT

RUN

```
const a = 10n;  
const b = 5;  
  
console.log(a + b);
```

এতে **TypeError** হবে।

দুইটিকে একই type করতে হবে:

13.54 JAVASCRIPT

RUN

```
console.log(a + BigInt(b));
```

অথবা:

13.54 JAVASCRIPT

RUN

```
console.log(Number(a) + b);
```

⚠ তবে বড় BigInt-কে Number-এ convert করলে precision loss হতে পারে।

13.55 BigInt Division

13.55 JAVASCRIPT

RUN

```
console.log(10n / 3n);
```

TEXT

3n

Decimal অংশ থাকে না।

13.56 BigInt Comparison

13.56 JAVASCRIPT

RUN

```
console.log(10n > 5);  
console.log(10n === 10);  
console.log(10n = 10);
```

TEXT

```
true  
false  
true
```

কারণ:

TEXT

```
10n !== 10
```

কিন্তু loose equality:

TEXT

```
10n = 10
```

true হতে পারে।

PART 04 — Intl API: INTERNATIONALIZATION, CURRENCY & LOCALIZATION (13.57 – 13.64)

13.57 Intl কী?

Intl মানে:

Internationalization

এটি বিভিন্ন দেশের:

```
Currency  
Number  
Date  
Time  
Language  
Plural
```

formatting-এর জন্য ব্যবহার করা হয়।

13.58 Currency Formatting

13.58 JAVASCRIPT

RUN

```
const price = 75000;  
  
const formatted = new Intl.NumberFormat(  
  "en-BD",  
  {  
    style: "currency",  
    currency: "BDT"  
  }  
)  
.format(price);  
  
console.log(formatted);
```

Output environment/browser অনুযায়ী formatting variation থাকতে পারে, যেমন:

TEXT

```
BDT 75,000.00
```

13.59 USD

13.59 JAVASCRIPT

RUN

```
const price = 1500;

const formatted = new Intl.NumberFormat(
  "en-US",
  {
    style: "currency",
    currency: "USD"
  }
).format(price);

console.log(formatted);
```

TEXT

\$1,500.00

13.60 EUR

13.60 JAVASCRIPT

RUN

```
const price = 1500;

console.log(
  new Intl.NumberFormat("de-DE", {
    style: "currency",
    currency: "EUR"
  }).format(price)
);
```

Output locale অনুযায়ী:

TEXT

1.500,00 €

13.61 Number Formatting

13.61 JAVASCRIPT

RUN

```
const number = 1234567.89;

console.log(
  new Intl.NumberFormat("en-US")
    .format(number)
);
```

TEXT

1,234,567.89

13.62 Bengali Locale

13.62 JAVASCRIPT

RUN

```
const number = 1234567.89;

console.log(
  new Intl.NumberFormat("bn-BD")
    .format(number)
);
```

এতে Bengali locale অনুযায়ী number formatting পাওয়া যাবে।

13.63 Date Formatting with Intl

13.63 JAVASCRIPT

RUN

```
const date = new Date("2026-09-16");

const formatted = new Intl.DateTimeFormat(
  "en-BD",
  {
    dateStyle: "full"
  }
).format(date);

console.log(formatted);
```

Possible output:

TEXT

Wednesday, September 16, 2026

13.64 Time Formatting

13.64 JAVASCRIPT

RUN

```
const date = new Date();

console.log(
  new Intl.DateTimeFormat("en-BD", {
    timeStyle: "medium"
  }).format(date)
);
```

PART 05 — REGEXP & STRING UTILITIES: PATTERNS, VALIDATION & MINI PROJECT (13.65 – 13.92)

13.65 RegExp কী?

RegExp

RegExp = **Regular Expression**

এটি text-এর মধ্যে pattern খুঁজতে/validate করতে ব্যবহৃত হয়।

যেমন:

TEXT

Email
Phone
Password
Username
URL
Search pattern

13.66 RegExp তৈরি করা

দুটি common syntax:

Literal

13.66 JAVASCRIPT

RUN

```
const pattern = /hello/;
```

Constructor

13.66 JAVASCRIPT

RUN

```
const pattern = new RegExp("hello");
```

13.67

test()

সবচেয়ে সহজ:

13.67 JAVASCRIPT

RUN

```
const pattern = /hello/;

console.log(
  pattern.test("hello world")
);
```

TEXT

true

13.68 Pattern না থাকলে

13.68 JAVASCRIPT

RUN

```
const pattern = /hello/;

console.log(
  pattern.test("good morning")
);
```

TEXT

false

13.69 Case Sensitivity

13.69 JAVASCRIPT

RUN

```
const pattern = /hello/;

console.log(
  pattern.test("Hello")
);
```

TEXT

false

কারণ lowercase `hello` এবং uppercase `Hello` আলাদা।

13.70

i

Flag

Case-insensitive:

13.70 JAVASCRIPT

RUN

```
const pattern = /hello/i;

console.log(
  pattern.test("Hello")
);
```

TEXT

true

13.71 g Flag

Global matching:

13.71 JAVASCRIPT

RUN

```
const pattern = /hello/g;
```

এটি multiple matches নিয়ে কাজ করতে সাহায্য করে।

13.72 Common RegExp Flags

Flag	Meaning
i	Case-insensitive
g	Global
m	Multiline
s	Dot matches newline
u	Unicode
y	Sticky

13.73 match()

13.73 JAVASCRIPT

RUN

```
const text = "JavaScript is great. JavaScript is powerful.";

console.log(
  text.match(/JavaScript/g)
);
```

TEXT

```
[
  "JavaScript",
  "JavaScript"
]
```

13.74 replace()

RegExp দিয়ে text replace:

13.74 JAVASCRIPT

RUN

```
const text = "I like cats";

const result = text.replace(
  /cats/,
  "dogs"
);

console.log(result);
```

TEXT

```
I like dogs
```

13.75 `replaceAll()`

13.75 JAVASCRIPT

RUN

```
const text = "cat cat cat";

const result = text.replaceAll(
  "cat",
  "dog"
);

console.log(result);
```

TEXT

dog dog dog

13.76 RegExp Character Classes

`[abc]`

a অথবা b অথবা c:

13.76 JAVASCRIPT

RUN

```
console.log(/[abc]/.test("apple"));
```

TEXT

true

`[0-9]`

যেকোনো digit:

13.76 JAVASCRIPT

RUN

```
console.log(/[0-9]/.test("Hello5"));
```

TEXT

true

`[a-z]`

lowercase letter:

13.76 JAVASCRIPT

RUN

```
console.log(/[a-z]/.test("123a"));
```

TEXT

true

13.77 Common RegExp Shortcuts

Pattern	Meaning
<code>\d</code>	Digit
<code>\D</code>	Not digit
<code>\w</code>	Word character
<code>\W</code>	Not word character
<code>\s</code>	Whitespace
<code>\S</code>	Not whitespace
<code>.</code>	Any character except line terminators

13.78 `\d` Example

13.78 JAVASCRIPT

RUN

```
console.log(/\d/.test("ABC5"));
```

TEXT

true

কারণ `5` একটি digit।

13.79 Quantifiers

+

এক বা একাধিক:

13.79 JAVASCRIPT

RUN

```
console.log(/\d+/.test("12345"));
```

TEXT

true

*

শূন্য বা একাধিক:

13.79 JAVASCRIPT

RUN

```
console.log(/\d*/.test(""));
```

TEXT

true

?

শূন্য বা এক:

13.79 JAVASCRIPT

RUN

```
console.log(/colou?r/.test("color"));  
console.log(/colou?r/.test("colour"));
```

দুটিই:

TEXT

true

13.80 `{n}`

ঠিক n বার:

13.80 JAVASCRIPT

RUN

```
console.log(/\d{4}/.test("2026"));
```

TEXT

true

13.81 `{n,m}`

কমপক্ষে n, সর্বোচ্চ m:

13.81 JAVASCRIPT

RUN

```
console.log(/\d{2,4}/.test("123"));
```

TEXT

true

13.82 Email Validation — Basic Example

13.82 JAVASCRIPT

RUN

```
const emailPattern =  
  /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
  
console.log(  
  emailPattern.test("ripon@example.com")  
);
```

TEXT

true

এটি basic validation pattern; production-grade email validation আরও nuanced হতে পারে।

13.83 Phone Validation — Basic Example

Bangladesh-এর 11-digit mobile number-এর basic pattern:

13.83 JAVASCRIPT

RUN

```
const phonePattern = /^01\d{9}$/;  
  
console.log(  
  phonePattern.test("01712345678")  
);
```

TEXT

true

13.84 Password Validation

ধরো password কমপক্ষে 8 characters:

13.84 JAVASCRIPT

RUN

```
const password = "Ripon123";  
  
console.log(  
  /^(.{8,})$/ .test(password)  
);
```

TEXT

true

আরও strong validation চাইলে uppercase, lowercase, digit, special character আলাদা করে enforce করতে হবে।

13.85



এবং



এগুলো খুব গুরুত্বপূর্ণ।

^ → শুরু
\$ → শেষ

Example:

13.85 JAVASCRIPT

RUN

```
/^hello$/
```

এর অর্থ:

পূরো string-টাই `hello` হতে হবে।

13.85 JAVASCRIPT

RUN

```
console.log(/^hello$/.test("hello"));  
console.log(/^hello$/.test("hello world"));
```

TEXT

```
true  
false
```

13.86

String

-এর কিছু গুরুত্বপূর্ণ Built-in Methods

যদিও String-এর অনেক method আমরা আগের chapters-এ দেখেছি, এখানে built-in utility হিসেবে গুরুত্বপূর্ণগুলো একসাথে রাখি।

`includes()`

13.86 JAVASCRIPT

RUN

```
const text = "JavaScript";  
  
console.log(text.includes("Script"));
```

TEXT

```
true
```

`startsWith()`

13.86 JAVASCRIPT

RUN

```
console.log(  
  "JavaScript".startsWith("Java")  
);
```

TEXT

```
true
```

`endsWith()`

13.86 JAVASCRIPT

RUN

```
console.log(  
  "JavaScript".endsWith("Script")  
);
```

TEXT

```
true
```

`repeat()`

13.86 JAVASCRIPT

RUN

```
console.log("Hi ".repeat(3));
```

TEXT

```
Hi Hi Hi
```

13.87 String.raw()

Special string representation পেতে:

13.87 JAVASCRIPT

RUN

```
const path = String.raw`C:\Users\Ripon\Desktop`;  
  
console.log(path);
```

TEXT

C:\Users\Ripon\Desktop

Template literal-এর escape processing-এর কিছু ক্ষেত্রে usefull

13.88 Number Conversion

String থেকে Number:

13.88 JAVASCRIPT

RUN

```
console.log(Number("100"));
```

TEXT

100

parseInt()

13.88 JAVASCRIPT

RUN

```
console.log(parseInt("100px"));
```

TEXT

100

parseFloat()

13.88 JAVASCRIPT

RUN

```
console.log(parseFloat("10.50px"));
```

TEXT

10.5

13.89 Number() বনাম parseInt()

13.89 JAVASCRIPT

RUN

```
console.log(Number("100px"));  
console.log(parseInt("100px"));
```

TEXT

NaN
100

কারণ:

```
Number()  
→ পুরো string-কে valid number হতে হবে  
  
parseInt()  
→ শুরু থেকে integer অংশ parse করতে পারে
```

13.90

toFixed()

Decimal places control করতে:

13.90 JAVASCRIPT

RUN

```
const price = 99.5678;

console.log(price.toFixed(2));
```

TEXT

99.57



toFixed() string return করে।

13.90 JAVASCRIPT

RUN

```
console.log(
  typeof price.toFixed(2)
);
```

TEXT

string

13.91 Real-Life Product Price

13.91 JAVASCRIPT

RUN

```
const price = 999.5678;

const finalPrice = Number(
  price.toFixed(2)
);

console.log(finalPrice);
```

TEXT

999.57

13.92 সবগুলো একসাথে — Mini Project

ধরো একটি product:

13.92 JAVASCRIPT

RUN

```
const product = {  
  name: "Laptop",  
  price: 75999.567,  
  discount: 10  
};
```

আমরা:

- Discount calculate করব
- 2 decimal-এ round করব
- Currency format করব
- Current date দেখাব

13.92 JAVASCRIPT

RUN

```
const product = {  
  name: "Laptop",  
  price: 75999.567,  
  discount: 10  
};  
  
const discountAmount =  
  product.price *  
  product.discount /  
  100;  
  
const finalPrice =  
  product.price - discountAmount;  
  
const formattedPrice =  
  new Intl.NumberFormat("en-BD", {  
    style: "currency",  
    currency: "BDT"  
  }).format(finalPrice);  
  
console.log("Product:", product.name);  
console.log("Price:", formattedPrice);  
console.log("Date:", new Date().toLocaleDateString());
```

Possible output:

```
Product: Laptop  
Price: BDT 68,399.61  
Date: 9/16/2026
```

Formatting browser/locale অনুযায়ী কিছুটা ভিন্ন হতে পারে।

MUST KNOW

🔥 Chapter 13 — Must Know

এইগুলো অবশ্যই ভালোভাবে জানতে হবে।

Date

JAVASCRIPT

RUN

```
new Date()  
Date.now()  
  
getFullYear()  
getMonth()  
getDate()  
getDay()  
getHours()  
  
toISOString()  
toLocaleDateString()
```

Math

JAVASCRIPT

RUN

```
Math.round()  
Math.floor()  
Math.ceil()  
Math.trunc()  
  
Math.max()  
Math.min()  
  
Math.random()  
  
Math.abs()  
Math.sqrt()  
Math.cbrt()  
Math.pow()  
Math.PI
```

Number

JAVASCRIPT

RUN

```
Number()  
Number.isInteger()  
Number.isNaN()  
Number.isFinite()  
Number.isSafeInteger()  
  
Number.MAX_SAFE_INTEGER
```

BigInt

JAVASCRIPT

RUN

```
12345678901234567890n
```

Intl

JAVASCRIPT

RUN

```
Intl.NumberFormat  
Intl.DateTimeFormat
```

RegExp

JAVASCRIPT

RUN

```
/pattern/
```

Important:

TEXT

```
test()  
match()  
replace()  
replaceAll()  
  
^  
$  
\\d  
\\w  
\\s  
+  
*  
?  
{ }  
[ ]
```

CHEAT SHEET

🗨 Quick Cheat Sheet

```
Date  
→ Date & Time  
  
Math  
→ Mathematical calculations  
  
Number  
→ Number validation/conversion  
  
BigInt  
→ Very large integers  
  
Intl  
→ International formatting  
  
RegExp  
→ Text pattern matching
```

Practice 1 — Date

আজকের:

```
Year
Month
Date
Day
Hour
Minute
```

console-এ print করো।

Practice 2 — Date Difference

```
Start: 2026-09-10
End: 2026-10-10
```

কত দিন difference বের করো।

Practice 3 — Random Number

1 থেকে 100-এর মধ্যে random integer generate করো।

Practice 4 — Maximum

JAVASCRIPT

RUN

```
const numbers = [10, 45, 22, 99, 67];
```

সবচেয়ে বড় এবং ছোট number বের করো।

Practice 5 — Currency

JAVASCRIPT

RUN

```
const price = 125000;
```

এটাকে Bangladesh currency format-এ দেখাও।

Practice 6 — BigInt

দুটি বড় integer:

TEXT

```
12345678901234567890
98765432109876543210
```

যোগ করো।

Practice 7 — RegExp

Check করো:

TEXT

```
01712345678
```

এটি basic Bangladesh mobile-number pattern follow করছে কিনা।

Practice 8 — Email

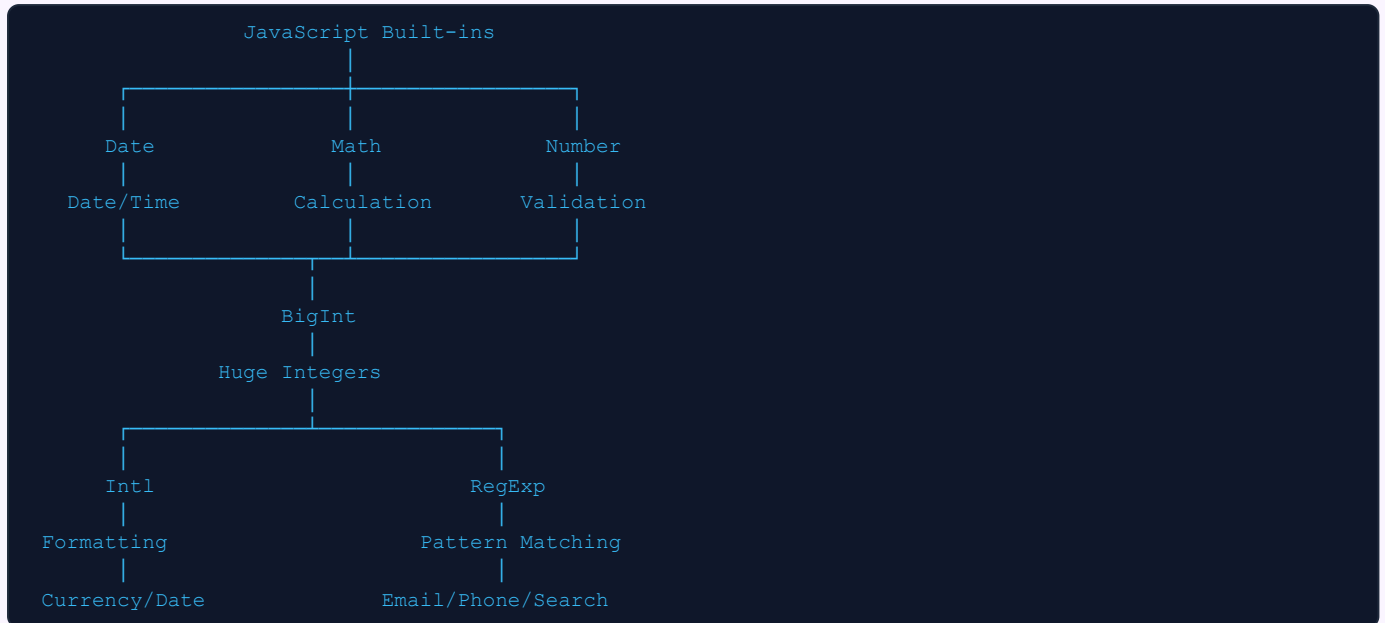
Check করো:

TEXT

```
ripon@example.com
```

এটি basic email pattern follow করছে কিনা।

পুরো Chapter-টাকে এভাবে মনে রাখো:



🔥 Chapter 13-এর সবচেয়ে গুরুত্বপূর্ণ বিষয়

JAVASCRIPT	RUN
<code>new Date()</code>	
JAVASCRIPT	RUN
<code>Math.random()</code>	
JAVASCRIPT	RUN
<code>Math.floor()</code>	
JAVASCRIPT	RUN
<code>Number.isInteger()</code>	
JAVASCRIPT	RUN
<code>Number.MAX_SAFE_INTEGER</code>	
JAVASCRIPT	RUN
<code>12345678901234567890n</code>	
JAVASCRIPT	RUN
<code>new Intl.NumberFormat()</code>	
JAVASCRIPT	RUN
<code>/^[^\s@]+@[^\s@]+\.[^\s@]+\$/</code>	

এগুলো ভালোভাবে আয়ত্ত করলে JavaScript-এর **date/time**, **mathematical calculation**, **number handling**, **large integers**, **international formatting** এবং **pattern validation**-এর ভিত্তি শক্ত হয়ে যাবে।